

北京航空航天大学__保研

北京航空航天大学·保研

提取函数调用及参数

- year: 2021

题目描述

给定一句C语言的非控制类（不包括 `if` / `while` 等）语句，例如 `a = find(b + c) + d` 这样的，输出其中所有的函数调用及传入的参数。

输入格式

输入为一句C语言的非控制类语句，语句中可能包含多个函数调用。

输出格式

输出语句中所有的函数调用及传入的参数，每个函数调用占一行。

函数名和参数之间用空格分隔，多个参数之间也用空格分隔。

如果没有参数，则只输出函数名。

输入样例

```
a=function(1,2+3,d)+functio()+functi(1+b+d);
```

输出样例

```
function 1 2+3 d
functio
functi 1+b+d
```

二进制序列操作

- year: 2022

题目描述

给定一个二进制数（该二进制数的长度可能非常长），可以对该二进制数进行以下4种操作： 1. 操作 `$ +`：将该二进制数加 1； 2. 操作 `-`：将该二进制数减 1； 3. 操作 `*`：将该二进制数乘 2； 4. 操作 `/`：将该二进制数整除 2；

给定一个二进制数以及包含多个操作的操作序列，求经过运算后的二进制数。

输入格式

输入包含3行： - 第一行包含2个正整数 `$ n` 和 `m`： `n` 表示输入的二进制数的长度， `m` 表示输入的操作序列所包含操作的个数。

- 第二行为1个字符串，用来表示输入的二进制数。

该字符串的长度为 `$ n`，且该字符串的每一个字符是 0 或者 1。

保证输入的二进制数不包含前导零。

- 第三行为1个字符串，用来表示输入的操作序列。

该字符串的长度为\$ m\$，且该字符串的每一个字符为 + 、 - 、 * 、 / \$中的其中之一。

保证运算过程不会出现负数。

输出格式

输出仅包含1行：一个字符序列，表示经过运算后的二进制数（不包含前导零）；如果运算结果为0，则输出0即可。

输入样例

```
3 4
101
+*-/
```

输出样例

```
101
```

图的连通性查询

- year: 2022

题目描述

给定图 G ， G 包含 n 个顶点。

现在有两种操作： 1. 在两个顶点间添加一条边（该边是双向的，且不保证两个顶点间只会有一条相连的边）。

2. 询问两个顶点之间是否存在互通的路径。

输入格式

第一行 2 个数，顶点个数 n （ $2 \leq n \leq 100000$ ），操作个数 m （ $1 \leq m \leq 1000000$ ）。

接下来 m 行，每行 3 个整数，分别为：操作种类 op （ $op = 1$ 为添加新边操作， $op = 2$ 为询问操作），第一个顶点编号 u （ $1 \leq u \leq n$ ），第二个顶点编号 v （ $1 \leq v \leq n$ ）。

（保证 u 和 v 不相等）

输出格式

对于每次询问操作（ $op = 2$ 时），输出一行， YES 表示互通， NO 表示不互通。

输入样例

```
3 3
2 1 2
1 1 2
2 1 2
```

输出样例

```
NO
YES
```

学生身高分配问题

- year: 2022

题目描述

有 N 名学生在排队做核酸，编号为 $1, 2, 3, \dots, N$ ，每名学生的身高都为整数。

当且仅当两名学生中间的学生身高都比他们矮时，两名学生方可看到对方。

现在，我们只知道最高的学生的身高是 H ，剩余学生的身高未知。

但是，我们还知道这群学生之中存在着 M 对关系，每对关系都指明了某两名学生 A 和 B 可以相互看见。

求每名学生的身高的最大可能值是多少。

输入格式

第一行输入整数 N, H, M ，数据用空格隔开。

接下来 M 行，每行两个整数 A 和 B ，代表学生 A 和学生 B 可以相互看见，数据用空格隔开。

输出格式

一共输出 N 行数据，每行输出一个整数。

第 i 行输出的整数代表第 i 名学生可能的最大身高。

输入样例

```
9 185 5
1 3
5 3
4 3
3 7
9 8
```

输出样例

```
185
184
185
183
184
184
185
185
185
```

数字塔

- year: 2022

题目描述

给定 $2n - 1$ 个数，可以按照以下方式构建一个数字塔：这 $2n - 1$ 个数排列在数字塔的最后一层；每上一层的数的个数减 2，同时该层每个位置的数的值是它正下方三个数的中位数。

```
    4 5 6
   3 4 5 6 7
  1 6 3 7 4 5 2
```

数字塔最顶层的数为该数字塔的代表数。

给定一个数字塔最后一层的 $2n - 1$ 个数，求出该数字塔的代表数。

输入格式

输入包括 2 行：- 第一行仅一个整数 n (n 的含义如上述所示)。

- 第二行包含 $2n - 1$ 个整数，表示数字塔最后一层的数。

输出格式

输出仅包含一行：该数字塔的代表数。

输入样例

```
4
1 6 3 7 4 5 2
```

输出样例

```
4
```

数据范围

所有数据在 int 范围之内，且 $2 \leq n \leq 100000$ 。

数组与堆

- year: 2022

题目描述

在各类实现堆的代码中，通常选择使用一维数组去模拟堆的结构。

如果在堆构建好之后，将这个一维数组输出，可以观察到它们有一些有趣的性质。

我们将小根堆（最小堆）构建完成后所对应的数列，称为 *小根堆数列*，大根堆（最大堆）构建完成后所对应的数列称为 *大根堆数列*。

堆是最重要的数据结构之一，堆的类型有2种：*最大堆*和*最小堆*。

对于*最大堆*，父结点的键值总是大于或等于任何一个子节点的键值。

对于*最小堆*，父结点的键值总是小于或等于任何一个子节点的键值。

通常堆的实现是用一维数组完成的。

给定一个长度为 n 的一维数组，请判断该一维数组是否实现了*最大堆*或者*最小堆*。

如果该一维数组实现了*最大堆*，请输出*Max heap*；如果该一维数组实现了*最小堆*，请输出*Min heap*；如果该一维数组既没实现*最大堆*，也没实现*最小堆*，请输出*Not a heap!*。

输入格式

第一行是一个正整数 T ，表示数据组数。

接下来有 T 组数据：每组数据包含 2 行，其中第 1 行为正整数 n ，表示一维数组的长度；第 2 行为 n 个正整数（每个正整数之间用空格隔开），表示该一维数组。

输出格式

对于每一组数据，输出 *Max heap*、*Min heap* 或者 *Not a heap!*，每个一行。

输入样例

```
3
6
2 3 4 5 6 7
3
5 2 1
9
3 1 4 1 5 9 2 6 5
```

输出样例

```
Min heap
Max heap
Not a heap!
```

数据范围

$T \leq 20$ $2 \leq n \leq 100000$ 数据保证不会有一维数组，既实现最大堆，又实现最小堆。

最大化求和

- year: 2022

题目描述

给定一个长度为 n 的数列 $A = a_1, a_2, \dots, a_i, \dots, a_n$ ，其中 $1 \leq i \leq n \leq 2000000$ ， $-50000 \leq a_i \leq 50000$ 。

当给定两个整数 x 和 y ($1 \leq x \leq y \leq n$)， $A[x : y]$ 表示数列 A 从下标 x 到下标 y 的子段；同时， $sum(x, y)$ 表示子段 $A[x : y]$ 中所有元素的和（即 $sum(x, y) = \sum_{i=x}^y a_i$ ）。

给定两个关于数列 A 的子段： $A[x_1 : y_1]$ 和 $A[x_2 : y_2]$ ，当 $x_1 \leq y_1 < x_2 \leq y_2$ 或者 $x_2 \leq y_2 < x_1 \leq y_1$ 时，则 $A[x_1 : y_1]$ 和 $A[x_2 : y_2]$ 这两个子段是不相交的。

现在给定一个正整数 m ，请找到数列 A 的 m 个互相不相交的子段 ($A[x_1 : y_1], A[x_2 : y_2], \dots, A[x_k : y_k], \dots, A[x_m : y_m]$)，使得 $\sum_{k=1}^m sum(x_k, y_k)$ 的值最大。

输入格式

输入包含 2 行：- 第 1 行为 2 个整数 m 和 n (m 和 n 的含义如上述所示)；- 第 2 行为 n 个整数， $a_1, a_2, \dots, a_n$ ，用来表示数列 A 。

输出格式

输出仅 1 行： $\sum_{k=1}^m sum(x_k, y_k)$ 的最大值。

输入样例

```
3 8
```

-1 -2 10 -3 7 -4 5 -9 3

输出样例

22

神奇2的幂

- year: 2022

题目描述

任意一个正整数 n 都可以表达成若干个 2 的幂相加。

例如，整数 9 有以下 10 种不同的与加数排序无关的表达方式：

1. $9 = 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$
2. $9 = 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1$
3. $9 = 2 + 2 + 1 + 1 + 1 + 1 + 1$
4. $9 = 2 + 2 + 2 + 1 + 1 + 1$
5. $9 = 2 + 2 + 2 + 2 + 1$
6. $9 = 4 + 1 + 1 + 1 + 1 + 1$
7. $9 = 4 + 2 + 1 + 1 + 1$
8. $9 = 4 + 2 + 2 + 1$
9. $9 = 4 + 4 + 1$
10. $9 = 8 + 1$

注意：在本题中， $9 = 8 + 1$ 和 $9 = 1 + 8$ 被认为是相同的与加数排序无关的表达方式。

因为它们的加数均为 8 和 1，仅加数的顺序不同。

输入格式

输入仅 1 行：一个正整数 n 。

输出格式

输出仅 1 行：正整数 n 的与加数排序无关的表达方式的个数（由于输出的数值可能会非常大，因此输出的数值需要对 1000007 取模）。

输入样例

9

输出样例

10

蓄水池储水问题

- year: 2022

题目描述

一个蓄水池由多个不同高度的平台构成。

给定一个 n 行 m 列的二维矩阵，表示该蓄水池由 $n \times m$ 个平台构成，同时也表明了每个平台的坐标位置。

将此二维矩阵中第 i 行第 j 列的数字记为 $h_{i,j}$ ，表示处在第 i 行第 j 列的平台的高度。

在该蓄水池中，所有平台之间形成的空间凹陷区域可以用于储存水，求该蓄水池总共可以储存的水量。

输入格式

第一行，2个正整数 n, m ，分别表示蓄水池所对应的二维矩阵的行数和列数。

接下来 n 行，每行 m 个非负整数，表示该行所包含的平台的高度。

输出格式

一行，仅包含1个非负整数，表示该蓄水池总共可以储存的水量。

输入样例

```
5 5
0 1 1 1 1
1 0 3 3 2
0 3 0 0 2
2 3 0 0 3
1 2 3 3 2
```

输出样例

9

数据范围

$$1 \leq n, m \leq 200 \quad 0 \leq h_{i,j} \leq 10^4$$

通讯录查询

- year: 2022

题目描述

小明同学有一个通讯录，通讯录中有 n 条记录。

对于每一条记录，分别有对方的姓名和电话号码。

假设通讯录中所有的姓名都不重复，请你帮小明同学写一个程序，以姓名为输入，查询该姓名对应的电话号码。

输入格式

第一行为两个整数 n 和 m ，其中 n 表示通讯录中记录的数量， m 表示查询的数量。

接下来的 n 行，每行 2 个字符串 $name$ 和 $number$ ，其中 $name$ 表示姓名， $number$ 表示 $name$ 对应的电话号码。

字符串 $name$ 和字符串 $number$ 均不含空格。

之后出现的 m 行，每行 1 个字符串 $query$ ，表示需要查询的姓名。

输出格式

对于每一次需要查询的姓名，输出对应的电话号码。

如果通讯录中没有该姓名，则输出 *NotFound*。

输入样例

```
2 3
alice 1234567
bob 7654321
alice
bob
catherine
```

输出样例

```
1234567
7654321
Not Found
```

老鼠找食物的最短路径

- year: 2023

题目描述

老鼠在寻找食物的过程中会记录移动的方向和距离。

移动的格式为 $x - y$ ，其中 x 表示方向（1 代表上，2 代表下，3 代表左，4 代表右）， y 表示移动的距离。

输入以 $0 - 0$ 结束，表示老鼠找到了食物。

老鼠在碰到死路时会原路返回到分叉路口，探索下一个方向。

需要求解老鼠原路返回的最佳路径，最佳路径的描述是“不走回头路”，即没有折返过程。

输入格式

输入为一系列 $x - y$ 形式的移动指令，以 $0 - 0$ 结束。

例如：

```
1-1 3-1 1-1 2-1 4-2 1-2 4-1 1-1 2-1 3-1 1-1 0-0
```

输出格式

输出为老鼠原路返回的最佳路径，以 $x - y$ 形式表示。

例如：

```
2-3 3-1 2-1
```

输入样例

```
1-1 3-1 1-1 2-1 4-2 1-2 4-1 1-1 2-1 3-1 1-1 0-0
```

输出样例

```
2-3 3-1 2-1
```

三叉树的最短路径

- year: 2025

题目描述

给定一个二叉树，小于 100 的值代表树叶，大于 100 的值为分支点。

建树的过程是水平方向建树。

输入格式：先输入 n ，代表有 n 组数据，接下来 n 行，每行输入 4 个数，第一个数代表根节点，接下来分别代表三个子节点，-1 代表子节点不存在，输入的顺序按照层次遍历的次序。

接下来，要求寻找叶子节点的最短路径，最短路径是指不经过重复的边。

输入方式，首先输入一个值 m ，代表 m 行，接下来 m 行输入 m 个叶子节点和对应的优先级，要求按优先级输出从上次到达的位置到该节点的最短路径，每条路径的最后一个节点要求输出目标叶子节点，最后要求回到根节点。

输入格式

- 第一行输入一个整数 n ，表示有 n 组数据。
- 接下来的 n 行，每行输入 4 个整数，表示一个节点及其三个子节点。
- 然后输入一个整数 m ，表示有 m 个查询。
- 接下来的 m 行，每行输入一个叶子节点和对应的优先级。

输出格式

- 对于每个查询，输出从上次到达的位置到该叶子节点的最短路径，路径的最后一个节点为目标叶子节点。
- 最后要求回到根节点。

输入样例

```
10
100 101 108 107
101 1 102 2
108 103 104 105
107 17 109 18
102 3 4 5
103 7 8 9
104 10 106 11
105 15 16 -1
109 19 20 21
106 12 13 14
5
8 1
14 3
16 2
5 0
19 4
```

输出样例

```
100 101 102 5
102 101 100 108 103 8
103 108 105 16
105 108 104 106 14
106 104 108 100 107 109 19
109 107 100
```

个位为1的素数

- year: 2025

题目描述

输入一个整数 n ($2 \leq n \leq 10000$)，要求输出所有从 1 到这个整数之间(不包括 1 和这个整数)个位为 1 的素数，如果没有则输出 -1。

输入格式

输入有多组数据。

每组一行，输入 n 。

输出格式

输出所有从 1 到这个整数之间(不包括 1 和这个整数)个位为 1 的素数(素数之间用空格隔开，最后一个素数后面没有空格)，如果没有则输出 -1。

输入样例

100

输出样例

11 31 41 61 71

函数调用深度分析

- year: 2025

题目描述

给你一个函数调用过程，求调用的最大嵌套深度所对应的函数名，并输出调用路径，同时输出这个函数父函数（调用者）的个数和被调用的次数。

同一个函数，如果有多条调用路径都可使其嵌套深度最深，则只输出第一条调用路径。

输入的数据中，若为 $1 funcName$ 的形式，则表示调用 $funcName$ 这个函数；若输入的是 0，则表示结束对嵌套深度最大的函数的调用。

输入格式

输入包含若干行，每行格式为 $1 funcName$ 或 0。

输出格式

输出包含三部分：1. 最大嵌套深度对应的函数名；2. 调用路径，格式为 $func1 - > func2 - > \dots - > funcN$ ；3. 父函数的个数和被调用的次数，格式为 $parentCount callCount$ 。

输入样例

```
1 main
1 input
0
1 area
0
```

```
1 findA
1 area
0
0
1 findB
1 get
0
0
0
```

输出样例

```
area main-findA 2 2
get main-findB 1 1
```

复杂计算器

- year: 2025

题目描述

升级版的四则运算模拟。

普通的四则运算是给出一个计算式字符串，其中所有运算符是 $+$, $-$, $*$, $/$, $($, $)$ 六种，运算数值是浮点数类型，保证运算式合法，要求输出他的计算结果。

升级版则是他的输入里面没有运算数值，而是各类变量名（如 *total*, *a* 这样的，可能有重复变量名出现多次，如 $(a + b + c)(a - b)$ 这种）。

然后下一行输入各个变量的浮点数数值，顺序按照各变量第一次出现的顺序给出。

要求输出四舍五入保留两位小数的运算结果。

输入格式

第一行输入一个计算式字符串，其中包含变量名和运算符。

第二行输入各个变量的浮点数数值，顺序按照各变量第一次出现的顺序给出。

输出格式

输出四舍五入保留两位小数的运算结果。

输入样例

```
(ab+cd+de)/(ab+cd)
5 5 5
```

输出样例

```
1.50
```

完全二叉排序树的层序遍历

- year: 2025

题目描述

给你一个整数序列，用这些数构成一个完全二叉排序树，输出此二叉树的层序遍历序列。

输入格式

输入的第一行是一个整数 n ，表示这个整数序列的长度。

输入的第二行包含 n 个整数，每个数代表完全二叉排序树的一个节点。

输出格式

输出由这 n 个整数构成的完全二叉排序树的层序遍历序列。

输入样例

```
18
56 987 -25 0 1021 -238 399 12 77 -1 72190 -25 -168 -41367 3218 12 0 -25
```

输出样例

```
12 -1 987 -25 0 77 3218 -238 -25 0 12 56 399 1021 72190 -41367 -168 -25
```

旋转矩阵

- year: 2025

题目描述

任意输入两个 9 阶以下矩阵，要求判断第二个是否是第一个的旋转矩阵，如果是，输出旋转角度 (0、90、180、270)，如果不是，输出 -1 。

输入格式

输入有多组数据。

每组数据第一行输入 n ($1 \leq n \leq 9$)，从第二行开始输入两个 n 阶矩阵。

每行两个数之间可以用任意个空格分隔。

行之间用回车分隔，两个矩阵间用任意的回车分隔。

输出格式

判断第二个是否是第一个的旋转矩阵，如果是，输出旋转角度 (0、90、180、270)，如果不是，输出 -1 。

如果旋转角度的结果有多个，则输出最小的那个。

输入样例

```
3
1 2 3
4 5 6
7 8 9
7 4 1
8 5 2
9 6 3
```

输出样例

```
90
```

等差素数序列

- year: 2025

题目描述

给定闭区间 $[a, b]$ ，要求输出连续的素数的等差序列，三个以上才算是序列。

例如 $[100, 200]$ 会输出 151 157 163，再例如输入 $[1, 100]$ 会有两个等差序列，3 5 7 和 47 53 59。

输出样式行末的空格保留。

输入格式

输入两个正整数 a 和 b ，其中 a 和 b 小于等于 10000。

输出格式

输出所有满足条件的等差素数序列，每个序列占一行，序列中的素数用空格分隔，行末保留空格。

输入样例

```
1 400
```

输出样例

```
3 5 7
47 53 59
151 157 163
167 173 179
199 211 223
251 257 263 269
367 373 379
```

连续合数段

- year: 2025

题目描述

给定区间 $[a, b]$ ，输出这个区间里最长的连续合数段。

输入格式

输入两个整数 a 和 b ， $a, b \leq 10000$ 。

输出格式

输出区间 $[a, b]$ 中最长的连续合数段。

输入样例

```
1 10
```

输出样例

```
8 9 10
```

